Space Empire

Choix techniques — Stack complète

Justification de chaque composant et des alternatives rejetées

Mars 2026

1. Vue d'ensemble

Couche	Technologie	Version
Langage	PHP	8.5
Framework	Symfony	8.x
Base de données	MariaDB	11.8.X *ou 12 si stable*
ORM	Doctrine ORM	3.x
Queue de messages	RabbitMQ	3.x
Temps réel	Mercure	latest
Templates	Twig	3.x
Compilation assets	Webpack Encore	latest
JS frontend	Stimulus	3.x
Authentification	Symfony Security (session)	—
Reverse proxy	Traefik	3.x
Conteneurisation	Docker + Docker Compose	latest
Serveur OS	Debian	12 ou 13

2. Backend PHP

2.1 PHP 8.5

~~PHP 8.3 est la version stable actuelle avec le meilleur support long terme. Les fonctionnalités utilisées dans le projet justifient cette version minimale.~~

- Enums natifs — utilisés pour UniverseStatus (pending, active, closed), Fleet statuses, etc.
- Readonly properties — utilisés dans les DTOs et Value Objects (readonly class CalendarDTO)
- Fibers — utilisées en interne par Symfony pour les opérations asynchrones
- Named arguments — utilisés dans les appels Doctrine et les factory methods
- Intersection types — utiles pour les interfaces de services

2.2 Symfony 8

Symfony 8 est le framework PHP le plus adapté pour un projet de cette complexité. Il fournit une architecture structurée, un écosystème de bundles matures et une LTS (Long Term Support) qui garantit la stabilité sur la durée du projet.

Composant	Rôle dans le projet
symfony/framework-bundle	Socle du framework — routing, DI container, kernel
symfony/security-bundle	Authentification par session, gestion des rôles, getUser(), voters
symfony/validator	Validation des DTOs avec attributs PHP (#[Constraints\...])
symfony/serializer	Normalisation des entités Doctrine en JSON pour les réponses API partielles
symfony/messenger	Queue de messages, dispatch RabbitMQ, DelayStamp pour les événements différés
symfony/amqp-messenger	Transport AMQP natif pour RabbitMQ — pas de bundle tiers nécessaire
symfony/mercure-bundle	Publication d'événements SSE vers le frontend (combat, construction, flotte...)
symfony/event-dispatcher	EventListeners — redirection si univers null/fermé, hooks applicatifs
symfony/console	Commandes CLI — app:universe:import, génération de carte, maintenance
symfony/yaml	Lecture des catalogues YAML (vaisseaux, bâtiments, technologies, univers)
symfony/finder	Scan des dossiers config/universes/*.json dans la commande d'import

Composant	Rôle dans le projet
symfony/asset	Gestion des chemins d'assets (CSS, JS, images) dans les templates Twig

> *Tous ces composants sont maintenus par l'équipe Symfony core. Pas de bundle communautaire fragile dans cette liste.*

Alternatives rejetées

> *Laravel — bon framework mais conventions très différentes de l'architecture visée. La structure Service/Manager/Repository/DTO ne s'y intègre pas aussi naturellement. Symfony est le choix correct pour une architecture explicite et maîtrisée.*

> *API Platform — trop opinionated. Génère automatiquement des endpoints REST/GraphQL qui court-circuitent l'architecture Controller → Service → Manager. On perdrait plus de temps à le contourner qu'à coder directement.*

> *FOS RestBundle / JMS Serializer — overhead inutile. Le serializer natif Symfony avec les groupes de normalisation couvre tous les besoins du projet.*

3. Base de données

3.1 MariaDB 11

MariaDB est un fork communautaire de MySQL, pleinement compatible avec le driver Doctrine. Il offre de meilleures performances sur les workloads de lecture intensive (cartes spatiales, listes de flottes, classements) et est activement maintenu.

- Compatibilité totale avec Doctrine ORM sans configuration spécifique
- JSON columns natives — utilisées pour le champ cargo des flottes et les settings d'univers
- Index full-text pour les recherches de joueurs et guildes
- Performances supérieures à MySQL sur les lectures avec beaucoup de jointures (flottes, planètes, univers)

Alternatives rejetées

> *PostgreSQL — excellent SGBD mais aucun avantage significatif sur MariaDB pour ce projet. Changer de SGBD en cours de route est coûteux, MariaDB est maîtrisé dans l'équipe.*

> *SQLite — uniquement pour les tests unitaires. Pas adapté à un jeu multi-joueurs en production.*

3.2 Doctrine ORM 3

L'ORM standard de l'écosystème Symfony. Il gère le mapping entités ↔ tables, les migrations et les requêtes complexes via QueryBuilder et DQL.

- Mapping par attributs PHP — #[ORM\Entity], #[ORM\Column]... lisible et colocalisé avec l'entité
- Lazy loading des associations — performances correctes par défaut, optimisable via JOIN explicites dans les repositories
- Unit of Work — Doctrine suit les changements en mémoire et optimise les requêtes SQL émises
- doctrine/doctrine-migrations-bundle — migrations versionnées, jouables en CI/CD et dans l'entrypoint Docker

4. Messaging et temps réel

4.1 RabbitMQ + Symfony Messenger

RabbitMQ est le broker de messages qui gère toutes les opérations différées du jeu. Symfony Messenger fournit l'abstraction PHP pour dispatcher et consommer ces messages.

Cas d'usage dans le projet :

- Fin de construction d'un vaisseau (DelayStamp = durée de construction calculée)
- Fin d'une recherche technologique
- Arrivée d'une flotte à destination
- Résolution d'un combat à T+X (timer fixe après la première arrivée)
- Fermeture d'un univers temporaire à ends_at

Pourquoi RabbitMQ plutôt qu'une alternative :

Solution	DelayStamp	Fiabilité	Verdict
RabbitMQ + plugin delayed	✓ natif	Haute	✓ **Retenu**
BDD + scheduler (cron)	✓ via cron	Très haute	Alternative valide
Redis Streams	Limité	Moyenne	✗ **Rejeté**
Cron seul	✗	Faible	✗ **Rejeté**

> *Le delayed message exchange plugin de RabbitMQ doit être activé explicitement dans le container Docker. Sans ce plugin, les DelayStamp Symfony ne fonctionnent pas correctement sur RabbitMQ.*

4.2 Mercure

Mercure est un protocole et un hub SSE (Server-Sent Events) qui permet au serveur de pousser des événements vers le navigateur sans polling. C'est la solution officielle de l'écosystème Symfony pour le temps réel.

Cas d'usage dans le projet :

- Notification d'attaque sur une planète du joueur
- Fin de construction d'un vaisseau — mise à jour de l'interface sans rechargement
- Arrivée d'une flotte alliée en renfort
- Fin d'une recherche technologique
- Changement de statut d'un univers (fermeture imminente)

Alternatives rejetées

> *WebSockets (Ratchet, Swoole) — complexité d'infrastructure sans bénéfice pour ce projet. Les notifications sont unidirectionnelles (serveur → client), SSE suffit largement. WebSockets seraient justifiés uniquement pour du chat temps réel bidirectionnel.*

> *Polling AJAX — solution de secours acceptable mais génère une charge inutile sur le serveur (requêtes toutes les N secondes même sans événement). Mercure est plus propre et plus efficace.*

5. Frontend

5.1 Twig

Twig est le moteur de template officiel de Symfony. Il génère le HTML côté serveur, ce qui simplifie l'architecture — pas d'API REST dédiée au rendu des pages, pas de framework SPA à maintenir.

Package	Rôle
symfony/twig-bundle	Intégration Twig dans le container Symfony
twig/twig	Moteur de template — héritage de layouts, macros, filtres
twig/extra-bundle	Filtres supplémentaires : date, intl, string, markdown, html

Alternatives rejetées

> *Vue.js / React en SPA — complexité disproportionnée pour le gain. Nécessite une API REST complète, gestion de l'état côté client, build JS lourd, CORS à gérer. Twig + Stimulus couvre 95% des besoins interactifs d'un jeu de gestion.*

5.2 Webpack Encore

Webpack Encore est le wrapper Symfony autour de Webpack. Il compile, minifie et versionne les assets CSS et JS. Il s'intègre proprement avec Twig via les helpers encore_entry_link_tags() et encore_entry_script_tags().

- Compilation Sass/CSS avec autoprefixer
- Tree shaking JS — seul le code utilisé est inclus dans le bundle final
- Content hash sur les fichiers compilés — cache busting automatique en production
- Hot Module Replacement en développement — rechargement automatique sans F5

Alternatives rejetées

> *Vite — excellent outil mais pas encore le standard Symfony. L'intégration avec symfony/asset et le bundle Twig est moins mature qu'Encore. Encore reste le choix le plus documenté dans l'écosystème Symfony.*

5.3 Stimulus

Stimulus est le framework JS minimaliste de l'écosystème Hotwire (créé par Basecamp/37signals, adopté par Symfony UX). Il ajoute du comportement JS à du HTML généré côté serveur — sans remplacer Twig par une SPA.

Principe : un controller Stimulus est attaché à un élément HTML via un attribut data-controller. Quand l'élément apparaît dans le DOM, Stimulus instancie le controller.

Package	Rôle
symfony/stimulus-bundle	Intégration Stimulus dans Symfony UX — auto-import des controllers
@hotwired/stimulus	Package JS Stimulus
@symfony/stimulus-bridge	Bridge entre Stimulus et Webpack Encore

Cas d'usage dans le projet :

- Écoute des événements Mercure SSE et mise à jour du DOM sans rechargement de page
- Formulaires interactifs (calcul de carburant en temps réel lors de l'envoi d'une flotte)
- Timers et compte à rebours (combat en cours, fin de construction)
- Tooltips, modales, menus déroulants — comportements UI sans jQuery

```
// Exemple — controller Stimulus pour les notifications Mercure
// assets/controllers/notification_controller.js

import { Controller } from '@hotwired/stimulus'

export default class extends Controller {
    static values = { mercureUrl: String }

    connect() {
        const url = new URL(this.mercureUrlValue)
        this.eventSource = new EventSource(url)

        this.eventSource.onmessage = (event) => {
            const data = JSON.parse(event.data)
            this.handleNotification(data)
        }
    }

    disconnect() {
        this.eventSource?.close()
    }

    handleNotification(data) {
        // mise à jour ciblée du DOM selon data.type
        // ex: 'combat_started', 'fleet_arrived', 'research_complete'
```

```
    }
}

{# Utilisation dans Twig #}
<div data-controller="notification"
     data-notification-mercure-url-value="{{ mercure_url }}">
</div>
```

Turbo — non retenu pour le MVP

> *Turbo (l'autre composant Hotwire) gère la navigation entre pages sans rechargement complet. Pour un jeu avec beaucoup d'état côté client (timers, SSE actifs, carte interactive), Turbo peut compliquer la gestion du cycle de vie des controllers Stimulus. Non retenu pour le MVP — peut être ajouté ultérieurement.*

6. Authentification

6.1 Symfony Security — session

L'authentification par session Symfony native est retenue. Le joueur se connecte via un formulaire Twig, Symfony crée une session côté serveur et maintient l'état de connexion via un cookie de session.

- Zéro dépendance supplémentaire — symfony/security-bundle est déjà inclus
- Intégration native avec getUser() dans les controllers et les templates Twig
- Voters Symfony pour les règles d'autorisation complexes (accès à un calendrier, gestion d'une guilde...)
- Guard authenticator pour le formulaire de connexion — standard Symfony bien documenté

```
# config/packages/security.yaml
security:
    firewalls:
        main:
            form_login:
                login_path: app_login
                check_path: app_login
            logout:
                path: app_logout
            remember_me:
                secret: '%kernel.secret%'
                lifetime: 604800   # 7 jours
```

Alternative rejetée

> *JWT (lexik/jwt-authentication-bundle) — adapté pour une API consommée par une SPA découplée ou une app mobile. Pas pertinent ici : le frontend est Twig, rendu côté serveur. JWT ajouterait une complexité inutile (gestion du token côté client, refresh token, CORS) sans aucun bénéfice.*

7. Infrastructure

7.1 Docker + Docker Compose

Tous les services sont conteneurisés. Docker Compose orchestre les containers en développement et en production.

Container	Contenu
php	PHP-FPM 8.5 + extensions (amqp, intl, pdo_mysql...) + code Symfony
nginx	Serveur web — sert les assets et proxifie vers PHP-FPM
mariadb	Base de données MariaDB 11
rabbitmq	Broker de messages avec le plugin delayed exchange activé
mercure	Hub Mercure pour les SSE
messenger_worker	Consumer Symfony Messenger — même image que php

> *Le container messenger_worker est une instance séparée de l'image PHP qui tourne uniquement php bin/console messenger:consume. Il consomme les messages RabbitMQ sans bloquer les requêtes HTTP.*

7.2 Traefik

Traefik est le reverse proxy qui expose les services vers l'extérieur. Il gère automatiquement les certificats SSL via Let's Encrypt et route le trafic vers le bon container selon le nom de domaine.

- Certificats SSL automatiques — Let's Encrypt sans configuration manuelle
- Routage par domaine — chaque univers peut potentiellement avoir son sous-domaine
- Dashboard de monitoring intégré — état des services en temps réel
- Labels Docker — configuration du routing directement dans docker-compose.yml

Alternative rejetée

> *Nginx comme reverse proxy — possible mais la gestion des certificats SSL est manuelle (Certbot + cron de renouvellement). Traefik automatise tout ça et s'intègre nativement avec Docker via les labels.*

7.3 Debian 12

Système d'exploitation du serveur hôte. Debian est choisi pour sa stabilité, sa longue durée de support (LTS 5 ans) et sa légèreté. C'est le système le plus répandu pour les serveurs de production PHP.

8. Utilitaires PHP

Package	Version	Utilisation
ramsey/uuid	^4.0	Génération des UUIDs pour les entités (Player, Fleet, Event...)
symfony/yaml	inclus Symfony	Lecture des catalogues YAML (vaisseaux, bâtiments, univers)
symfony/finder	inclus Symfony	Scan des dossiers de configuration (import des univers)
phpunit/phpunit	^11.0	Tests unitaires et fonctionnels
symfony/test-pack	inclus Symfony	Helpers de test Symfony (WebTestCase, KernelTestCase)

8.1 Sur Redis

Redis n'est pas inclus dans la stack MVP. Les catalogues YAML (vaisseaux, bâtiments, technologies) sont des services Symfony singleton — ils sont chargés une seule fois en mémoire par process PHP-FPM et réutilisés sur toutes les requêtes. Redis deviendrait pertinent dans ces cas :

- Cache de sessions distribué — si plusieurs instances PHP tournent en parallèle (hors scope MVP)
- Cache de résultats de calculs lourds — classements, statistiques globales d'univers
- Rate limiting — si on veut limiter les actions par joueur sans toucher la BDD

> *À ajouter en post-MVP si des problèmes de performance sont observés. Ne pas l'ajouter par anticipation.*